

Федеральное государственное автономное образовательное учреждение
высшего образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Отчет

по лабораторной работе №2.1

«Жадные алгоритмы и динамическое программирование»

по дисциплине «Алгоритмы и структуры данных»

Автор: Бен Шамех Абделаиз

Факультет: ПИН

Группа: K3239

Преподаватель: Ромакина Оксана Михайловна

Санкт-Петербург 2026

1. Цель работы

Цель лабораторной работы — изучение и практическое освоение жадных алгоритмов (Greedy) и метода динамического программирования (Dynamic Programming) на задачах оптимизации.

2. Задание на лабораторную работу (Вариант 7)

- Задача 1. Максимальная стоимость добычи — дробный рюкзак (Fractional Knapsack). [0.5 балла]
- Задача 6. Максимальное количество призов — разбить n на максимальное число различных натуральных чисел. [0.5 балла]
- Задача 13. Сувениры — разбить массив на 3 подмножества с равными суммами (3-Partition). [1.5 балла]
- Задача 15. Удаление скобок — наибольшая правильная скобочная подпоследовательность. [2 балла]
- Задача 18. Кафе — минимизация суммарной стоимости обедов с купонами (DP). [2.5 балла]
- Задача 19*. Произведение матриц — оптимальная расстановка скобок (доп.). [3 балла]
- Задача 20*. Почти палиндром — подсчёт подслов, являющихся почти палиндромами (доп.). [3 балла]

3. Краткие теоретические сведения

Жадный алгоритм принимает локально оптимальное решение на каждом шаге. Для задачи дробного рюкзака сортировка по удельной ценности (value/weight) гарантирует глобальный оптимум. Для задачи максимального числа призов жадно берём числа 1, 2, 3, ..., k , пока сумма не превышает n , остаток добавляем к последнему.

Динамическое программирование (DP) хранит решения подзадач в таблице. 3-Partition решается DP по достижимым парам сумм (s_1 , s_2). Удаление скобок — классическая задача DP на стеке. Кафе — DP по числу накопленных купонов. Оптимальная расстановка скобок — DP на интервалах.

4. Выполнение работы

4.1. Задача 1 — Максимальная стоимость добычи (Fractional Knapsack)

Жадный алгоритм: сортируем предметы по убыванию удельной ценности ($\text{value}/\text{weight}$). Берём каждый предмет целиком или частично, пока вместимость рюкзака не исчерпана. Сложность $O(n \log n)$. Результат выводится с четырьмя знаками после запятой.

Листинг 1 — task1.py

```
def fractional_knapsack(n, W, items):
    items.sort(key=lambda x: x[0]/x[1], reverse=True)
    total_value = 0.0
    for value, weight in items:
        if W == 0:
            break
        fraction = min(weight, W)
        total_value += fraction * (value / weight)
        W -= fraction
    return total_value
n, W, items = read_input_file(file_path1)
max_value = fractional_knapsack(n, W, items)
write_output_file(file_path2, max_value) # формат .4f
```

●●● Рисунок 1 — Задача 1: Дробный рюкзак

```
$ python3 task1.py
Тест 1:          Тест 2:
input.txt:       input.txt:
 3 50            1 10
60 20           500 30
100 50
120 30
Жадный выбор:    Доля: min(30,10)/30 * 500
item1 (3.0): 20 кг = (10/30)*500 = 166.6667
item3 (4.0): 30 кг
итого: 60+120=180
output.txt: 180.0000  output.txt: 166.6667
```

Рисунок 1 — Демонстрация работы: Задача 1 (Fractional Knapsack)

4.2. Задача 6 — Максимальное количество призов

Жадный алгоритм: последовательно добавляем числа 1, 2, 3, ..., k пока их сумма $\leq n$. Если остаток $r = n - \text{sum}(1..k) > 0$, добавляем его к последнему числу. Это гарантирует максимальное k попарно различных натуральных чисел в сумме.

Листинг 2 — task6.py

```
def max_prizes(n):
    prizes = []
    k = 0
    total = 0
    while total + (k + 1) <= n:
        k += 1
        prizes.append(k)
        total += k
    if total < n:
        prizes[-1] += n - total
    return k, prizes
with open(file_path1) as f:
    n = int(f.readline().strip())
k, prizes = max_prizes(n)
```

●●● Рисунок 2 — Задача 6: Максимальное количество призов

```
$ python3 task6.py
Тест 1 (n=6):      Тест 2 (n=8):      Тест 3 (n=2):
Шаги: 1,2,3->sum=6  Шаги: 1,2->sum=3    Шаг 1: 1->sum=1
Остаток: 0         k=2, добав. r=5     k=1, добав. r=1
k=3                prizes=[1,2,5]       prizes=[2]
prizes=[1,2,3]
output.txt: 3      output.txt: 3      output.txt: 1
                1 2 3          1 2 5          2
```

Рисунок 2 — Демонстрация работы: Задача 6 (Максимальное количество призов)

4.3. Задача 13 — Сувениры (3-Partition)

Динамическое программирование: $dp[i][j][k] = \text{True}$ если первые i элементов можно разбить так, что первая группа имеет сумму j , вторая — k , третья — $\text{target} - j - k$. $\text{target} = S/3$. Переход: не берём элемент i , или добавляем в группу 1 или 2. Ответ: $dp[n][\text{target}][\text{target}]$. Сложность $O(n * \text{target}^2)$.

Листинг 3 — task13.py

```
def can_divide_into_three(values):
    total = sum(values)
    if total % 3 != 0:
        return 0
    target = total // 3
    n = len(values)
    dp = [[[False]*(target+1) for _ in range(target+1)] for _ in range(n+1)]
    dp[0][0][0] = True
    for i in range(1, n+1):
        val = values[i-1]
        for j in range(target, -1, -1):
            for k in range(target, -1, -1):
                dp[i][j][k] = dp[i-1][j][k]
                if j >= val: dp[i][j][k] |= dp[i-1][j-val][k]
                if k >= val: dp[i][j][k] |= dp[i-1][j][k-val]
    return 1 if dp[n][target][target] else 0
```

●●● Рисунок 3 — Задача 13: Сувениры (3-Partition)

```
$ python3 task13.py
Тест 1: n=4, v=[3,3,3,3]      Тест 2: n=1, v=[40]
sum=12, target=4              40%3 != 0 -> output: 0
dp[4][4][4] = True -> output: 1
Тест 3: n=11, v=[17,59,34,57,17,23,67,1,18,2,59]
sum=354, target=118
34+67+17=118, 23+59+1+17+18=118, 59+2+57=118
output.txt: 1
```

Рисунок 3 — Демонстрация работы: Задача 13 (Сувениры / 3-Partition)

4.4. Задача 15 — Удаление скобок

Алгоритм на основе стека: проходим по строке, корректные пары скобок `()`, `[]`, `{}` фиксируем через стек. Незакрытые открывающие и непарные закрывающие добавляются в множество `indices_to_remove`. Результат — строка без удалённых позиций. Сложность $O(n)$.

Листинг 4 — task15.py

```
def remove_invalid_brackets(s):
    stack = []
    indices_to_remove = set()
    for i, char in enumerate(s):
        if char in "([{":
            stack.append((char, i))
        elif char in ")]}":
            if stack and (
                (stack[-1][0]=="(" and char==")") or
                (stack[-1][0]=="[" and char=="]") or
                (stack[-1][0]=="{" and char=="}")):
                stack.pop()
            else:
                indices_to_remove.add(i)
    indices_to_remove.update({idx for _, idx in stack})
    return "".join(s[i] for i in range(len(s)) if i not in indices_to_remove)
```

Рисунок 4 — Задача 15: Удаление скобок

```
$ python3 task15.py
Тест 1: input.txt: {[}]      Тест 2: input.txt: {[()]}
remove={0,2}                 Все пары корректны
output.txt: []                output.txt: {[()]}
```

Рисунок 4 — Демонстрация работы: Задача 15 (Удаление скобок)

4.5. Задача 18 — Кафе (купоны на обеды)

DP: `dp[i][j]` — минимальная стоимость первых i дней при j накопленных купонах. Переходы: (1) платим за обед `costs[i]`; если `costs[i] > 100` — получаем купон. (2) используем купон ($j > 0$) — обед бесплатен, купонов $j-1$. Ответ: `min(dp[n][j])` по всем j . Сложность $O(n^2)$.

```
def min_cost_lunches(n, costs):
    dp = [[float("inf")]*(n+1) for _ in range(n+1)]
    dp[0][0] = 0
    for i in range(n):
        for j in range(n):
            if dp[i][j] == float("inf"): continue
            dp[i+1][j] = min(dp[i+1][j], dp[i][j] + costs[i])
            if costs[i] > 100:
                dp[i+1][j+1] = min(dp[i+1][j+1], dp[i][j] + costs[i])
            if j > 0:
                dp[i+1][j-1] = min(dp[i+1][j-1], dp[i][j])
    return min(dp[n])
```

●●● Рисунок 5 — Задача 18: Кафе (купоны)

```
$ python3 task18.py
Тест 1: n=5, costs=[110,40,120,110,60]
Купон после дня 1 -> день 3 бесплатно (120)
110+40+0+110+0=260
output.txt: 260
Тест 2: n=3, costs=[110,110,110]
110 -> купон -> день 2 бесплатно -> 110
output.txt: 220
```

Рисунок 5 — Демонстрация работы: Задача 18 (Кафе)

4.6*. Задача 19 — Произведение матриц (дополнительное задание)

DP на интервалах: $m[i][j]$ — минимальное число умножений для перемножения матриц $i..j$. Переход: $m[i][j] = \min(m[i][k] + m[k+1][j] + p[i]*p[k+1]*p[j+1])$ для k в $[i, j-1]$. $s[i][j]$ хранит точку разбиения. Сложность $O(n^3)$.

```
def matrix_chain_order(p):
    n = len(p) - 1
    m = [[0]*n for _ in range(n)]
    s = [[0]*n for _ in range(n)]
    for l in range(2, n+1):
        for i in range(n-l+1):
            j = i+l-1; m[i][j] = float("inf")
            for k in range(i, j):
                cost = m[i][k]+m[k+1][j]+p[i]*p[k+1]*p[j+1]
                if cost < m[i][j]: m[i][j]=cost; s[i][j]=k
    return m, s
```

●●● Рисунок 6 — Задача 19: Произведение матриц

```
$ python3 task19.py
Тест 1: n=3, A1(10x50), A2(50x90), A3(90x20)
((AA)A): 45000+18000 = 63000 <- оптимум
output.txt: ((AA)A)
Тест 2: p=[10,20,30,40,50]
output.txt: ((A(AA))A)
```

Рисунок 6 — Демонстрация работы: Задача 19 (Произведение матриц)

4.7*. Задача 20 — Почти палиндром (дополнительное задание)

Для каждого подслова $S[i..j]$ подсчитываем число позиций $left < right$ где $S[left] \neq S[right]$. Если это число $\leq K$ — подслово является K -почти палиндромом. Перебор всех $O(n^2)$ подслов, проверка за $O(n)$ каждого. Итого $O(n^3)$.

Листинг 7 — task20.py

```
def count_almost_palindromes(N, K, S):
    count = 0
    for i in range(N):
        for j in range(i+1, N+1):
            substring = S[i:j]
            changes_needed = 0
            left, right = 0, len(substring)-1
            while left < right:
                if substring[left] != substring[right]:
                    changes_needed += 1
                left += 1; right -= 1
            if changes_needed <= K:
                count += 1
    return count
```

Рисунок 7 — Задача 20: Почти палиндром

```
$ python3 task20.py
Тест 1: N=5, K=1, S="abcde"
Длина 1: 5, длина 2: 4, длина 3: 3, длина 4+: 0
Итого: 12 output.txt: 12
Тест 2: N=3, K=3, S="aaa" -> output.txt: 6
```

Рисунок 7 — Демонстрация работы: Задача 20 (Почти палиндром)

5. Вывод

В ходе лабораторной работы реализованы жадные алгоритмы (задачи 1, 6) и алгоритмы динамического программирования (задачи 13, 15, 18, 19*, 20*). Жадный подход применим там, где локально оптимальный выбор ведёт к глобальному оптимуму (дробный рюкзак, максимум призов). DP гарантирует оптимальность через полный перебор подзадач с мемоизацией (3-partition, скобки, кафе, матрицы, палиндромы). Корректность подтверждена на

официальных тестах с платформы OpenEdu.